

Today's agenda

Best time to buy and sell stock

Best time to buy and sell stock II

Best time to buy and sell stock with transaction fee

Best time to buy and sell stock with cooldown

Best time to buy and sell stock III



AlgoPrep

Best time to buy and sell stock

- maximize your Profit in one transaction.
- before you can sell, you need to buy first.

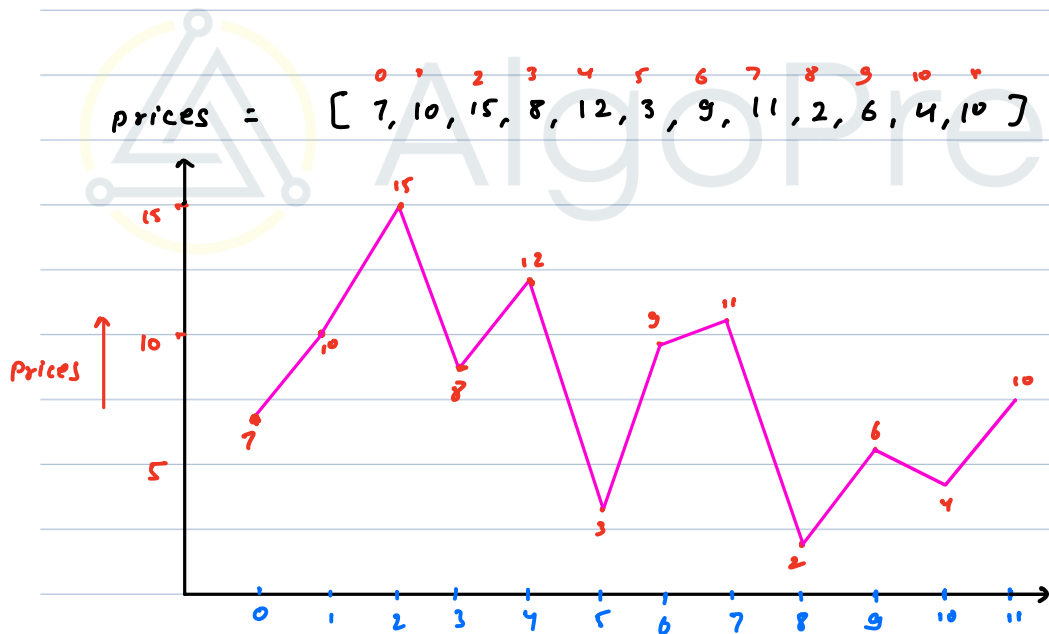
prices = [10, 7, 5, 2]



min BP = 10 > 5 > 2

prices = [10, 7, 5, 2, 3, 5, 8, 4]

(-3, -2, -5, 1, 3, 6, 2) → max



Pseudo code:

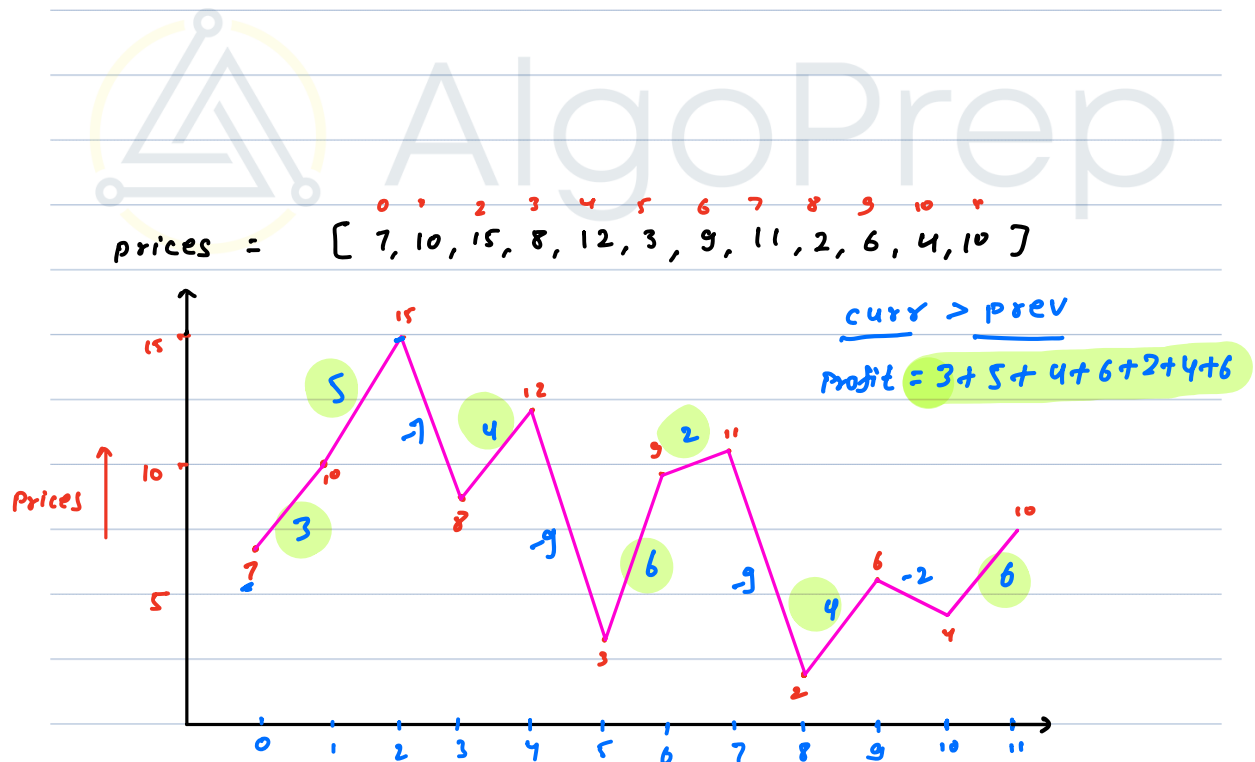
```
public int maxProfit ( int[] Prices ) {  
    int maxP = 0;  
    int minBP = prices[0];  
    for (int i=1; i < n; i++){  
        int tP = Prices[i] - minBP;  
        if (tP > maxP){  
            maxP = tP;  
        }  
        minBP = Math.min(minBP, Prices[i]);  
    }  
    return maxP;  
}
```

Best time to buy and sell stock II

On each day, you may decide to buy and/or sell the stock. (infinite transaction allowed)

• before you can sell, you need to buy first.

eg.: prices = [5, 8, 12, 6, 15]



Pseudocode

```
public int maxProfit ( int [] Prices) {  
    int tP = 0;  
    for (int i=1; i<n; i++) {  
        if ( Prices[i] - Prices[i-1] > 0) {  
            tP += Prices[i] - Prices[i-1];  
        }  
    }  
    return tP;  
}
```

Best time to buy and sell stock with transaction fee

• you may complete as many transaction as you like, but you need to pay the transaction fee for each transaction.

prices = [7, 11, 15, 8, 13, 3, 9, 11, 2, 6], fee = 3

		buy	sell
→ 0	7	-7 b_0	0
→ 1	11	-7 b_0	1 b_0, s_1
→ 2	15	-7 b_0	5 b_0, s_2
→ 3	8	-3 b_0, s_2, b_3	5 b_0, s_2
→ 4	13	-3	7 b_0, s_2, b_3, s_4
→ 5	3	4	7
→ 6	9	4	10
7	11		buy → (buy, sell - prices[i])
8	2		sell → (sell, prices[i] + buy - fee)
9	6		

BS BS B
B

BS BS BS
B

Pseudo Code

```
Public int maxProfit ( int[] Prices, int fee)
```

```
int buy = -Prices[0];
```

```
int sell = 0
```

```
for( int i=1; i<n; i++) {
```

```
int nbuy = Math.max( buy, sell - Prices[i]);
```

```
int nsell = Math.max( sell, Prices[i] + buy - fee);
```

```
buy = nbuy;
```

```
sell = nsell;
```

```
}
```

```
return sell;
```

```
}
```

```
public int maxProfit (int[] Prices, int fee)
```

```
int buy = -Prices[0];
```

```
int sell = 0
```

```
for (int i=1; i<n; i++){
```

```
int nbuy = Math.max (buy, sell-Prices[i]);
```

```
int nsell = Math.max (sell, Prices[i]+buy-fee);
```

```
buy = nbuy;
```

```
sell = nsell;
```

```
}
```

```
return sell;
```

```
{
```

```
→ 0 7
```

```
→ 1 11
```

```
2 15
```

```
→ 3 8
```

```
→ 4 13
```

```
→ 5 3
```

```
6 9
```

```
7 11
```

```
8 2
```

```
9 6
```

buy

-7

-7

-7

-3 (buy 2, 3)

-3

sell

0

1

5 (buy 2)

5

7

break fill 9:25

Best time to buy and sell stock with cooldown

- you may complete as many transaction as you like, but after you sell your stock, you cannot buy stock on the next day. (cooldown one day)

prices = [7, 11, 15, 8, 13, 3, 9, 11, 2, 6]

	buy	sell	cooldown
0 7	-7	0	0
1 11	-7	4	0
→ 2 15	-7	8	4
→ 3 8	-4	8	8
4 13			
5 3			
6 9			
7 11			
8 2			
9 6			

$buy = (buy, cool - price(i)) \rightarrow \max$

$sell = (sell, price(i) + buy) \rightarrow \max$

$cool = (cool, sell) \rightarrow \max$

Pseudo Code

```
public int maxProfit ( int [] Prices) {  
    int buy = - Prices[0], sell = 0, cool = 0;  
  
    for ( int i=1; i<n; i++) {  
        int nbuy = Math.max( buy, cool - prices[i]);  
        int nsell = Math.max( sell, prices[i] + buy);  
        cool = Math.max( cool, sell);  
        buy = nbuy;  
        sell = nsell;  
    }  
    return sell;  
}
```

Pseudo Code

```
public int maxProfit(int[] Prices){
    int buy = - Prices[0], sell=0, cool=0;
```

```
    for (int i=1; i<n; i++){
```

```
        int nbuy = Math.max(buy, cool - Prices[i]);
```

```
        int nsell = Math.max(sell, Prices[i] + buy);
```

```
        cool = Math.max(cool, sell);
```

```
        buy = nbuy;
```

```
        sell = nsell;
```

```
    }
    return sell;
```

3

		buy	sell	cooldown
	0 7	-7	0	0
	1 11	-7	4	0
	2 15	-7	8	4
	3 8	-4	8	8
	4 13	-4	9	8
	5 3	5	9	9
	6 9	5	14	9
	7 11	5	16	14
	8 2	12	16	16
	9 6	12	18	16

$n_{buy} = (buy, cool - Price[i]) \rightarrow \max$

$n_{sell} = (sell, Prices[i] + buy) \rightarrow \max$

$cool = (cool, sell) \rightarrow \max$

Best time to buy and sell stock III

- find the maximum profit you can achieve. you may complete at most two transactions.
- you must sell the stock before you buy again.

eg:- $[\overset{0}{3}, \overset{1}{15}, \overset{2}{19}, \overset{3}{8}, \overset{4}{13}]$ BSBS (2)

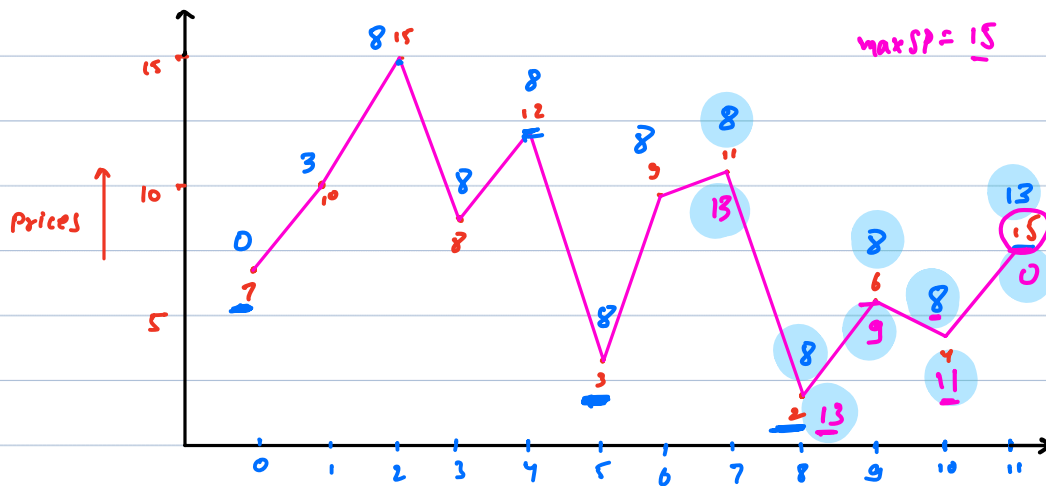
$$16 + 5 = \underline{\underline{21}}$$

single trans
left

single trans
right

eg:- $[\overset{0}{3}, \overset{1}{15}, \overset{2}{19}, \overset{3}{8}, \overset{4}{20}, \overset{5}{18}, \overset{6}{25}]$
 $\underline{16} \quad 17 = \underline{\underline{33}}$

prices = [7, 10, 15, 8, 12, 3, 9, 11, 2, 6, 4, 15]



AlgoPrep

Used Code

```
public int maxProfit (int[] Prices) {  
    int left() = new int(n);  
    int right() = new int(n);  
  
    int maxP = 0;  
    int minBP = prices(0);  
    for (int i = 1; i < n; i++) {  
        int tP = prices(i) - minBP; →  
        if (tP > maxP) {  
            maxP = tP;  
        }  
        minBP = Math.min(minBP, prices(i));  
        left[i] = maxP;  
    }  
    maxP = 0;  
    int maxSP = prices[n-1];  
    for (int i = n-2; i ≥ 0; i--) {  
        int tP = maxSP - prices(i);  
        if (tP > maxP) { maxP = tP }  
        maxSP = Math.max(maxSP, prices(i));  
        right[i] = maxP;  
    }  
}
```

```
int ans = 0;
```

```
for(int i = 0; i < n; i++) {
```

```
    ans = Math.max(ans, left[i] + right[i]);  
}
```

```
return ans;
```

3



AlgoPrep